VINUNIVERSITY

Memory Systems for Agents

AICB-P2T3 · Ngày 17 · Chương 4 — Agent Nâng Cao

Giảng viên

VinUniversity · Phase 2 · Track 3 · Tuần 4

HÃY SUY NGHĨ...

"Tại sao agent của bạn quên mọi thứ sau mỗi conversation — và làm sao fix nó đúng cách?"

Giữ câu hỏi này trong đầu suốt buổi học hôm nay

Nội dung vận hành

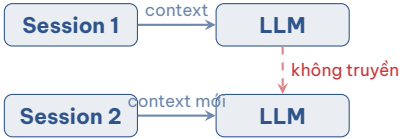
1. Tại sao Agent “quên”?
2. Context Engineering Framework
3. Cognitive Memory Model — 4 loại Memory
4. Implementation Deep-Dive
5. Frameworks chuyên dụng & Privacy
6. Demo & Thực hành

01

Tại sao Agent “quên”?

Context window có giới hạn — và hầu hết agent không có bộ nhớ ngoài

Agent hiện tại — Stateless by default

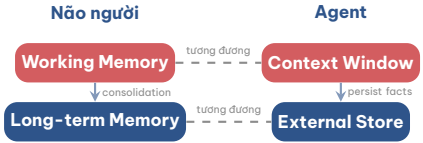


Mỗi session bắt đầu từ zero

- LLM **không có persistent state** — mỗi API call là một request độc lập
- User nói "tôi thích Python" ở session 1 → session 2 agent **không nhớ**
- Conversation dài >50 turns → **hit context limit**

Lưu ý: Đây là vấn đề #1 khi deploy agent thực tế: user kỳ vọng agent "nhớ" — nhưng nó không.

Analogy: Bộ nhớ Agent giống não người



Context Window = RAM — Nhanh, tạm thời, giới hạn dung lượng (~128K tokens)

External Store = Ổ cứng — Chậm hơn, bền vững, gần như vô hạn (Redis, Vector DB)

Agent cần cả hai: **fast access** cho conversation hiện tại + **persistent storage** cho knowledge qua sessions

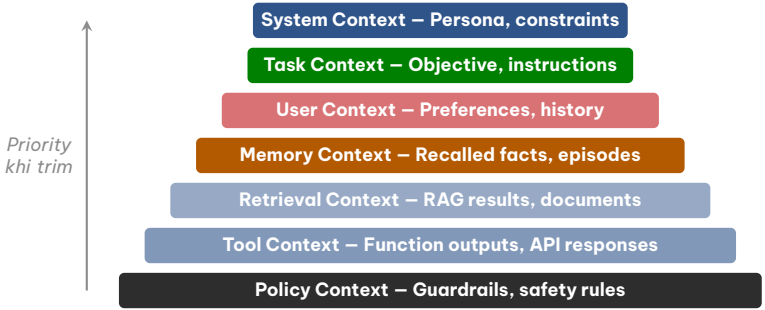
VINUNIVERSITY

02

Context Engineering Framework

7 layers of context — quản lý những gì agent “thấy”

7 Context Layers — Kiến trúc thông tin cho Agent



Khi gần token limit: **trim từ dưới lên**. Policy context trim cuối cùng (safety không bao giờ bỏ).

Lưu ý: Conflict resolution: user preference mâu thuẫn policy constraint → policy **luôn thắng**. Cần explicit rules trong system design.

Token Budget — Phân bổ context window

10%	4%	3%	3%
Short-term memory	Long-term facts	Episodic memory	Semantic knowledge

> Phần còn lại dành cho system prompt, task instructions, tool outputs, và output generation. Vượt 20% → context bị nhiễu, accuracy giảm.

VINUNIVERSITY

03

Cognitive Memory Model – 4 loại Memory

Short-term, Long-term, Episodic, Semantic

4 loại Memory — Cognitive Model cho AI Agents

Tạm thời | *Bền vững*

Cá nhân | *Tri thức*

Short-term (Working)
Context window buffer
Nhanh, tạm thời, ~128K tokens

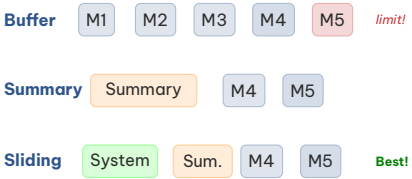
Long-term (Declarative)
Redis, PostgreSQL
User prefs, facts qua sessions

Episodic
Log trải nghiệm có thứ tự
"Lần trước tôi đã làm gì?"

Semantic
Embeddings + Vector DB
Domain knowledge retrieval

Working memory ↔ Short-term | Declarative memory ↔ Long-term | Episodic & Semantic tương tự tên gọi

Short-term Memory — Context Window Management

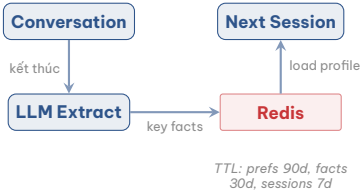


3 strategies chính:

1. **Buffer**: giữ tất cả — đơn giản nhưng hit limit sau ~50 turns
2. **Summary**: LLM tóm tắt history cũ — ổn định nhưng tốn thêm LLM calls
3. **Sliding window**: system + summary + last K turns — *best tradeoff cho production*

> Short-term memory nên chiếm tối đa **10% context window**. Trim khi vượt — keep recent N tokens, discard oldest.

Long-term Memory với Redis — Persistent Cross-Session

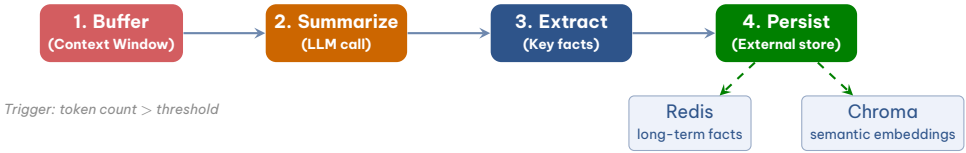


Ý tưởng:

- Sau mỗi conversation, LLM **extract key facts** rồi store vào Redis với TTL
- Session mới: load user profile vào system prompt *trước* khi user nói

Hash: preferences (language, style)
Set: facts ("biết Python, học ML")
List: session history (recent)
Tất cả **O(1) reads** — production-ready

Memory Management Flow — Buffer → Summarize → Store



Chỉ persist sau **task completion** — không write giữa chừng để tránh inconsistent state

Lưu ý: Conflict resolution: long-term fact mâu thuẫn short-term info → **recency wins**, flag for review.

04

Implementation Deep-Dive

Code-level: LangGraph nodes cho mỗi memory type

LangGraph Memory State — Code-Level

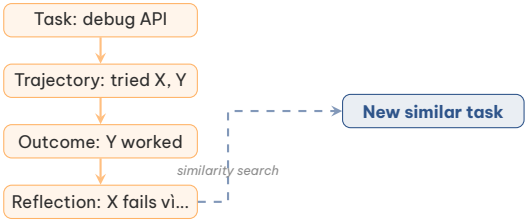
```python
class MemoryState(TypedDict):
    messages: list[BaseMessage]
    user_profile: dict        # long-term
    episodes: list[dict]      # episodic
    semantic_hits: list[str]  # semantic
    memory_budget: int        # tokens left

# Memory router: ọchn ạloi phù ghp
def retrieve_memory(state):
    query = state["messages"][-1].content
    return {
        "user_profile": redis.hgetall(uid),
        "episodes": find_similar(query, k=3),
        "semantic_hits": chroma.query(query),
    }
```

Integration pattern:

1. Node `load_memory`: đọc 3 loại memory khi bắt đầu
2. Inject vào system prompt theo priority
3. Node `save_memory`: ghi khi kết thúc

1. Short-term (gần nhất)
2. Long-term facts (user prefs)
3. Relevant episodes
4. Semantic knowledge

Episodic Memory — Learning từ Past Trajectories

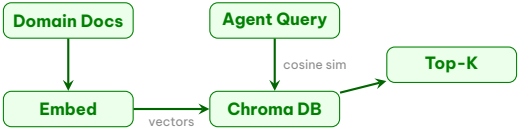


Lưu tuple mỗi episode:

- `(task, trajectory, outcome, reflection)`
- Agent biết: "approach X đã fail vì Y trong task tương tự"

LRU: xóa episode ít dùng nhất
Importance decay: score giảm theo thời gian
Consolidation: merge episodes tương tự
Voyager-style: extract reusable strategy → skill library

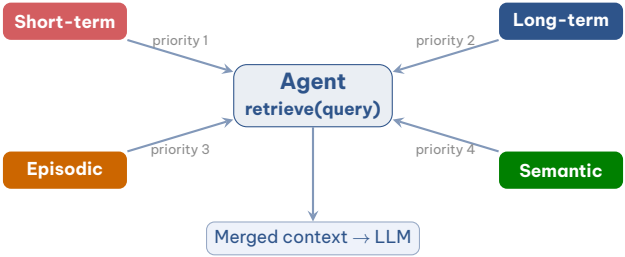
Semantic Memory — Vector DB cho Knowledge Retrieval



- Encode domain knowledge → embeddings → Chroma/Pinecone
- Query = task description → cosine similarity → top-k chunks
- Agent discover facts mới → add vào DB với metadata (`source`, `confidence`, `timestamp`)

> Agent tự mở rộng knowledge base qua interactions — incremental knowledge growth

Memory Architecture — Combining All 4 Types



Lưu ý: Unified interface: `retrieve(query, types=["all"])` trả về merged context từ cả 4 loại memory, đã trim theo token budget.

VINUNIVERSITY

05

Frameworks chuyên dụng & Privacy

Mem0, Zep — và khi nào dùng framework có sẵn

Mem0 & Zep — Managed Memory Layers

- Auto-classify memory types
- Smart retrieval: relevance + recency ranking
- Claim: **90% token reduction**, 91% faster retrieval
- API-first, nhanh go-to-market

- Entity extraction + progressive summarization
- Tự build user knowledge graph qua sessions
- Multi-level summaries: turn → session → cross-session
- Giảm context size tối ưu

Tiêu chí	Mem0 / Zep	Custom (Redis + Chroma)
Setup time	Nhanh (API)	Chậm (build from scratch)
Control	Hạn chế	Full control
Khi nào dùng	MVP, go-to-market	Production, đặc thù domain

Quyền riêng tư, Bảo mật & GDPR

Privacy-by-Design — Mặc định **không lưu PII**. User phải explicit opt-in trước khi agent ghi nhớ thông tin cá nhân.

Right to be Forgotten — User yêu cầu xóa → xóa **tất cả** memory entries liên quan → confirm deletion.

Lưu ý: Federated Forgetting: trong multi-agent system, deletion request phải **propagate** đến tất cả agents có copy.

- ☑ Data minimization
- ☑ Purpose limitation
- ☑ Storage limitation (TTL)
- ☑ Consent management
- ☑ Deletion verification

06

Demo & Thực hành

Xem agent nhớ user preferences qua 3 sessions

Agent nhớ User Preferences qua 3 Sessions

1. **Session 1:** User nói "tôi thích Python, không thích Java" → agent ghi vào Redis
2. **Session 2** (new process): Agent load memory → proactively suggest Python solution mà không cần hỏi lại
3. **Session 3:** Agent recall episode "user bị confused async/await" → tự thêm explanation
4. So sánh: agent có memory vs không memory — response relevance, user satisfaction

Lab #17

Mục tiêu: Build Multi-Memory Agent với LangGraph

Deliverable: Agent với full memory stack + benchmark report: so sánh agent có/không memory trên 10 multi-turn conversations

Thời gian: 2 giờ

Lab 17 — Các bước thực hành

1. **Implement 4 memory backends**: ConversationBufferMemory (short-term), Redis (long-term), JSON episodic log, Chroma (semantic)
2. **Build memory router**: chọn memory type phù hợp dựa trên query intent — user preference vs factual recall vs experience recall
3. **Context window management**: auto-trim khi gần limit, priority-based eviction theo 4-level hierarchy
4. **Benchmark**: so sánh agent có/không memory trên 10 multi-turn conversations — đo response relevance, context utilization, token efficiency

> GitHub repo + benchmark report: bảng so sánh metrics, memory hit rate analysis, token budget breakdown

Tổng kết — Key Takeaways

VINUNIVERSITY

Những ý chính cần nhớ sau buổi học hôm nay

1. Không có "one size fits all" — production agent cần ít nhất **short-term + long-term**, thêm episodic/semantic tùy use case
2. Memory retrieval quality quyết định agent quality — bad retrieval = irrelevant context = wrong answer
3. Memory write-back cần careful design: nhớ gì, khi nào ghi, xử lý conflict ra sao, TTL bao lâu
4. Privacy không phải afterthought — GDPR compliance cần thiết kế từ đầu (Privacy-by-Design)

Tiếp theo & Bài tập

Ngày 18: Production RAG

"Agent đã có memory, tiếp theo cần knowledge retrieval tốt hơn — tại sao RAG pipeline demo chạy tốt nhưng production chỉ đạt 60%?"

- Hoàn thành Lab 17: Multi-Memory Agent + benchmark
- Đọc: Anthropic "Building Effective Agents" (mục Context Engineering)

Hỏi & Đáp

Memory nào là “must-have” cho production agent?
Khi nào thì dùng framework (Mem0, Zep) vs tự build?

Cảm ơn!

AICB-P2T3 · Ngày 17 · Memory Systems for Agents

github.com/vinuni-aicb

Liên hệ: instructor@vinuni.edu.vn